

ITF23019: Parallel and Distributed Programming

Report for Lab 4: ForkJoin Framework

Name: Emil Berglund

GitHub: EmilB04

Exercise 1: Review and run code

1.1: Review code to see how to implement tasks in Forkjoin

Demo Recursive Task:

Task.java extends RecursiveTask<Double> and implements compute() using the usual pattern: if the assigned index range is small, it sums prices directly in a loop; if it's large, it splits the range into two subtasks, forks one, computes the other, then combines the two results.

The pool usage is in Main.java. Main creates a big product list, builds one root Task for the full range, runs it in ForkJoinPool.commonPool() via pool.execute(task), waits until it's done, then reads the final sum from the task (task.get()) and shuts the pool down.

Demo Recursive Action:

Task.java extends RecursiveAction and overrides compute(). If the assigned range is small, it updates prices directly; otherwise, it splits the range into two new Tasks and runs them in parallel using fork() + join(), or invokeAll, shown as an alternative.

The pool usage is in Main.java. It uses ForkJoinPool.commonPool() to get a pool, submits the task with pool.execute(task), waits until the task is done, then calls pool.shutdown().

1.2: Run code and include screenshots (5 Screenshots)

Demo_recursive_action (2 screenshots):

```

Parallel-distributed-programming/lab4 on main [?]
> cd /home/emilb_linux/dev/V2026/Parallel-distributed-programming/lab4
1-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages
/User/workspaceStorage/fb5583bc2f03566a4d426fb6c1dc4c/Parallel-distributed-programming/lab4
ActionMain.Main
*****
Main: Parallelism: 15
Main: Active Threads: 13
Main: Task Count: 125
Main: Steal Count: 6
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 120
Main: Steal Count: 6
*****
*****
Main: Parallelism: 15
Main: Active Threads: 13
Main: Task Count: 99
Main: Steal Count: 16
*****
*****
Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 71
Main: Steal Count: 16
*****
*****
Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 87
Main: Steal Count: 22
*****
*****
Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 96
Main: Steal Count: 22
*****
*****

```

```

Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 98
Main: Steal Count: 22
*****
*****
Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 102
Main: Steal Count: 22
*****
*****
Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 97
Main: Steal Count: 22
*****
*****
Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 96
Main: Steal Count: 26
*****
*****
Main: Parallelism: 15
Main: Active Threads: 10
Main: Task Count: 75
Main: Steal Count: 31
*****
*****
Main: Parallelism: 15
Main: Active Threads: 14
Main: Task Count: 92
Main: Steal Count: 45
*****
*****
Main: Parallelism: 15
Main: Active Threads: 11
Main: Task Count: 23
Main: Steal Count: 115
*****
Main: The process has completed normally.
Main: End of the program.

```

Parallel-distributed-programming/lab4 on  main [?]

Demo_recursive_task (3 screenshots):

```

Parallel-distributed-programming/lab4 on @ main |
> cd /home/emilb_linux/dev/V2026/Parallel-distrib
1-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInEx
/User/workspaceStorage/fb5583bc2f03566a4d426fb6c
TaskMain.Main
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 107
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 117
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 110
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 108
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 117
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 118
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 120
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 115
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 98
Main: Steal Count: 0
*****
*****

```

```

Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 91
Main: Steal Count: 0
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 99
Main: Steal Count: 6
*****
*****
Main: Parallelism: 15
Main: Active Threads: 12
Main: Task Count: 100
Main: Steal Count: 14
*****
*****
Main: Parallelism: 15
Main: Active Threads: 12
Main: Task Count: 90
Main: Steal Count: 14
*****
*****
Main: Parallelism: 15
Main: Active Threads: 12
Main: Task Count: 92
Main: Steal Count: 14
*****
*****
Main: Parallelism: 15
Main: Active Threads: 12
Main: Task Count: 96
Main: Steal Count: 14
*****
*****
Main: Parallelism: 15
Main: Active Threads: 12
Main: Task Count: 108
Main: Steal Count: 14
*****
*****
Main: Parallelism: 15
Main: Active Threads: 12
Main: Task Count: 83
Main: Steal Count: 14
*****
*****

```

```

Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 113
Main: Steal Count: 20
*****
*****
Main: Parallelism: 15
Main: Active Threads: 15
Main: Task Count: 99
Main: Steal Count: 20
*****
*****
Main: Parallelism: 15
Main: Active Threads: 13
Main: Task Count: 91
Main: Steal Count: 21
*****
*****
Main: Parallelism: 15
Main: Active Threads: 13
Main: Task Count: 92
Main: Steal Count: 38
*****
Main: The process has completed normally.
Sum of the prices: 100000000.000000
Main: End of the program.

```

Parallel-distributed-programming/lab4 on  main [?]

Exercise 2: Investigate threadshold and level of parallelism.

- (10 pts) Create a project and name it Exercise for this exercise.
 - o **DONE – Created project with the name «Exercise 2»**
- (10 pts) Complete the implementation of Task class and Main class.
 - o **DONE – ToDo list has been resolved, and program runs.**
- (10 pts) Run your code with different values of Threshold and Level of Parallelism and add screenshots to your report.
 - o **DONE – Screenshots below.**

```

Parallel-distributed-programming/lab4/exercise2 on main [!?]
> java -cp bin Main 10000 4
Threshold: 10000 - Parallel level: 4 - Number of processors: 16
The number of even numbers is 4999958
Running time is 39 ms

Parallel-distributed-programming/lab4/exercise2 on main [!?]
> java -cp bin Main 20000 6
Threshold: 20000 - Parallel level: 6 - Number of processors: 16
The number of even numbers is 5000130
Running time is 44 ms

Parallel-distributed-programming/lab4/exercise2 on main [!?]
> java -cp bin Main 60000 12
Threshold: 60000 - Parallel level: 12 - Number of processors: 16
The number of even numbers is 5000008
Running time is 77 ms

Parallel-distributed-programming/lab4/exercise2 on main [!?]
> 

```

```

Parallel-distributed-programming/lab4/exercise2 on main [!?]
> java -cp bin Main 120000 16
Threshold: 120000 - Parallel level: 16 - Number of processors: 16
The number of even numbers is 5000078
Running time is 52 ms

Parallel-distributed-programming/lab4/exercise2 on main [!?]
> java -cp bin Main 240000 16
Threshold: 240000 - Parallel level: 16 - Number of processors: 16
The number of even numbers is 5000011
Running time is 58 ms

Parallel-distributed-programming/lab4/exercise2 on main [!?]
> |

```

- (10 pts) How the running time of the program varies with the above two arguments?
 - o **threshold = when to stop splitting work.**
 Smaller threshold → more (smaller) tasks → better core usage, but more overhead.
 Bigger threshold → fewer (bigger) tasks → less overhead but can leave cores idle.
 - o **parallel_level = how many threads the ForkJoinPool uses.**
 Higher (up to your CPU cores) → usually faster.
 Too high (above cores) → little gain or slower (thread overhead).
- (10 pts) What happens when you set paral level higher than the number of processors cores available in your computer? Run the code with different settings and include the screenshots of the output in your report to support your arguments.
 - o For this task I did it on my portable MacBook which has 8 processor cores.

- For this task I also asked Claude Sonnet 4.5 for some help - creating a nice and clear overview in the console.

```

echo "======" && echo "COMPREHEN
"Array size: 10,000,000 elements" && echo "Threshold: 100,000" && e
n 1 2 4 8 16 32 64 128; do echo "--- Parallel Level = $i ---"; java
=====
COMPREHENSIVE PARALLELISM LEVEL TEST
System: 8 processor cores available
Array size: 10,000,000 elements
Threshold: 100,000
=====

--- Parallel Level = 1 ---
Threshold: 100000 - Parallel level: 1 - Number of processors: 8
The number of even numbers is 4999986
Running time is 65 ms

--- Parallel Level = 2 ---
Threshold: 100000 - Parallel level: 2 - Number of processors: 8
The number of even numbers is 5000057
Running time is 39 ms

--- Parallel Level = 4 ---
Threshold: 100000 - Parallel level: 4 - Number of processors: 8
The number of even numbers is 4999938
Running time is 32 ms

--- Parallel Level = 8 ---
Threshold: 100000 - Parallel level: 8 - Number of processors: 8
The number of even numbers is 4999937
Running time is 48 ms

--- Parallel Level = 16 ---
Threshold: 100000 - Parallel level: 16 - Number of processors: 8
The number of even numbers is 5000013
Running time is 41 ms

--- Parallel Level = 32 ---
Threshold: 100000 - Parallel level: 32 - Number of processors: 8
The number of even numbers is 5000035
Running time is 48 ms

--- Parallel Level = 64 ---
Threshold: 100000 - Parallel level: 64 - Number of processors: 8
The number of even numbers is 5000145
Running time is 60 ms

--- Parallel Level = 128 ---
Threshold: 100000 - Parallel level: 128 - Number of processors: 8
The number of even numbers is 5000077
Running time is 49 ms

```

Observations:

Firstly, the program never crashed or failed to work.

Secondly, performance does not improve beyond the number of physical cores. The best performance was around 4-8 parallel level. Beyond 8 cores there are slight performance degradations.

- (10 pts) Which values of Threshold and Level of Parallelism give you the (likely) shortest running time?
 - **Threshold:**
 - Too small (10,000): More task subdivision overhead
 - Too large (200,000+): Less parallelism opportunity
 - Sweet spot: 25,000-50,000 balances subdivision and parallel execution
 - **Level of Parallelism:** With a system of 8 cores, level 4 parallelism performs best. This is due to less thread coordination and reduced context switching.
- (10 pts) Use one AI tool to complete the implementation of Task class and Main class. Show the difference between the solution generated by AI and your own solution.
 - **NOT DONE**